Bauhaus-Universität Weimar

JOHNS HOPKINS UNIVERSITY

From FEM to Neural Operators: Scientific Machine Learning for Computational Mechanics

Discretizing the Weak Form

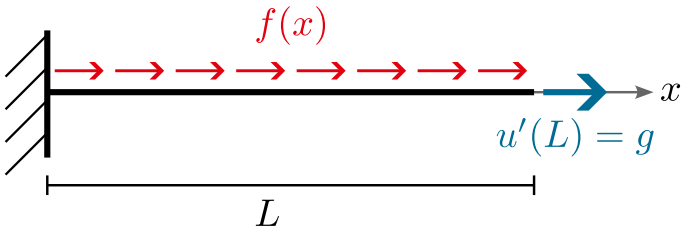
Dr-Ing Jorge Alberto López Zermeño

Bauhaus Spring School 2026

Starting Point: The Weak Form

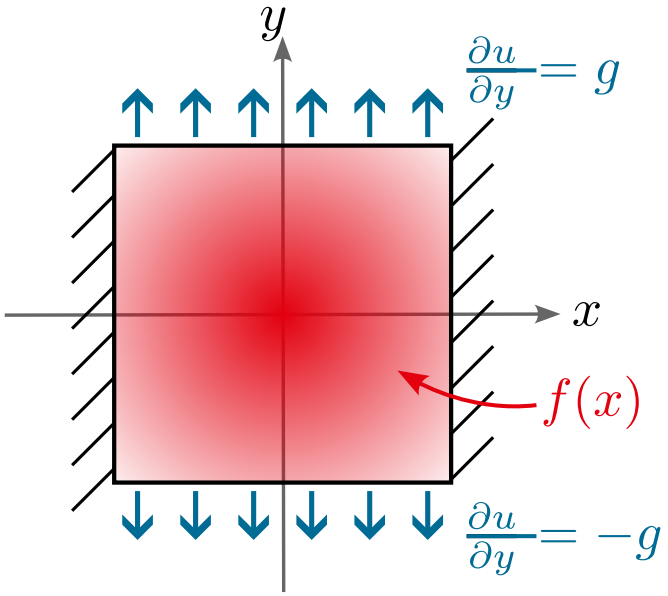
Weak form of the Poisson equation

1D:



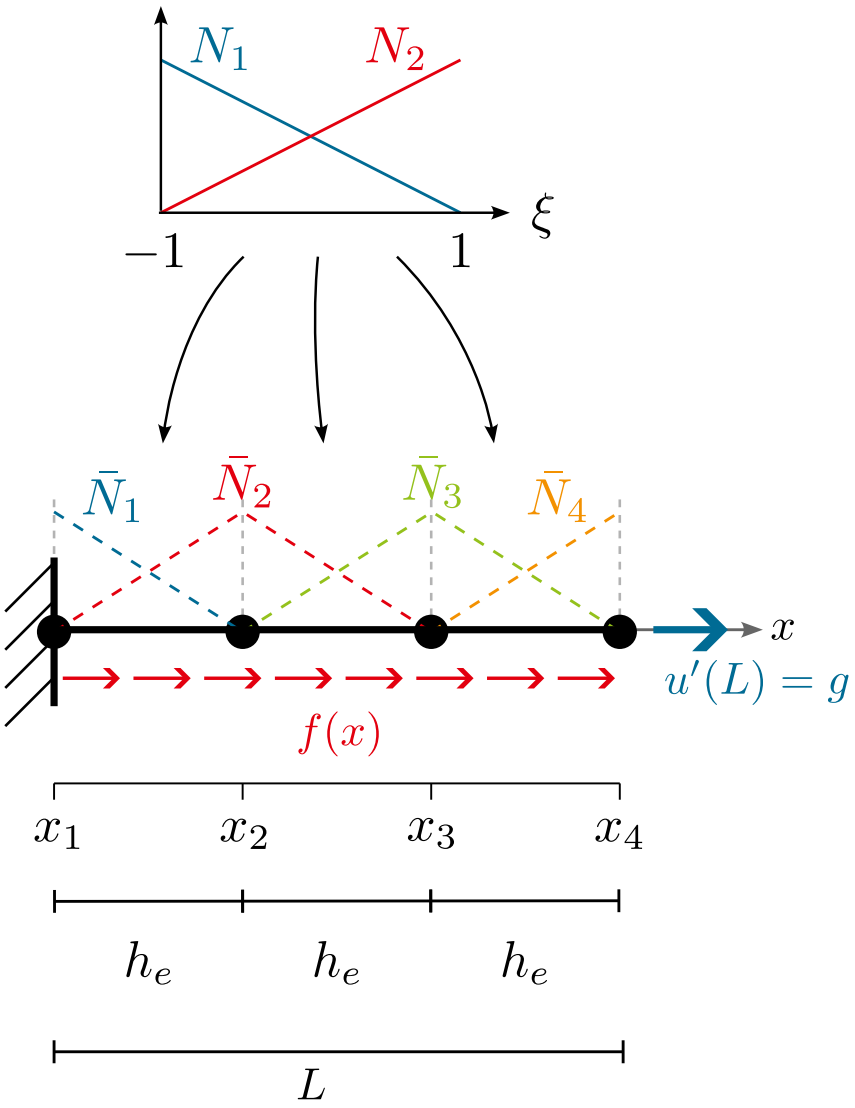
$$\int_0^L u'(x)\, v'(x)\, dx = \int_0^L f(x)\, v(x)\, dx$$

2D:



$$\int_\Omega \nabla u \cdot \nabla v \, d\Omega = \int_\Omega f\, v \, d\Omega$$

Discrete Spaces (1D)



Global discretization:

$$u(x) = \sum_{i=1}^{n} \bar{N}_i(x)\, u_i,$$

$$v(x) = \sum_{i=1}^{n} \bar{N}_i(x)\, v_i.$$

- $\bar{N}_i$ are assembled from shape functions on each element that share node i.
- Local support.

Element level:

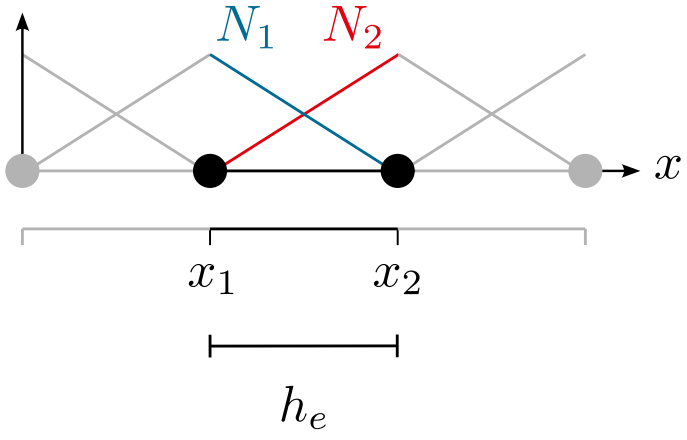
$$u_e(x) = \sum_{i=1}^{n} N_i(x)\, u_i^e,$$

$$v_e(x) = \sum_{i=1}^{n} N_i(x)\, v_i^e.$$

From Weak Form to Discrete (Matrix) Form in 1D

Weak form:

$$\int_{he} u'(x)\, v'(x)\, dx = \int_{he} f(x)\, v(x)\, dx$$



Discretizing the gradients:

$$u'_e(x) = \sum_{i=1}^{n} N'_i(x)\, u_i^e, \quad v'_e(x) = \sum_{i=1}^{n} N'_i(x)\, v_i^e.$$

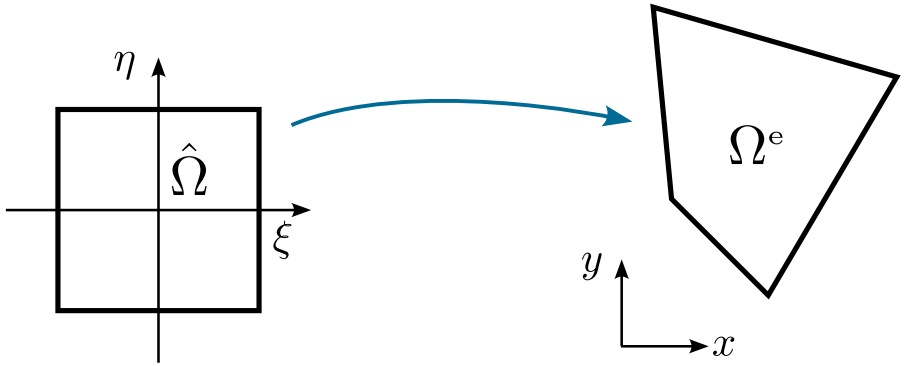
$$\sum_{i=1}^{n}\sum_{j=1}^{n} \int_{he} N'_i(x)\, u_i^e N'_j(x)\, v_j^e\, dx = \sum_{j=1}^{n} \int_{he} f(x)\, N'_i(x)\, v_i^e\, dx$$

$$K_{ij} = \int_0^L N'_i(x)\, N'_j(x)\, dx, \qquad f_i = \int_0^L f(x)\, N_i(x)\, dx$$

$$\Rightarrow \quad \mathbf{K}\,\mathbf{u} = \mathbf{f}.$$

From Weak Form to Discrete (Matrix) Form in 2D

$$\nabla N_a = \mathbf{J}^{-1}\,\nabla_\xi N_a, \quad \mathbf{J} = \begin{bmatrix} \dfrac{\partial x}{\partial \xi} & \dfrac{\partial y}{\partial \xi} \\ \dfrac{\partial x}{\partial \eta} & \dfrac{\partial y}{\partial \eta} \end{bmatrix}$$



$$\int_\Omega \nabla u \cdot \nabla v \, d\Omega = \int_\Omega f\, v \, d\Omega$$

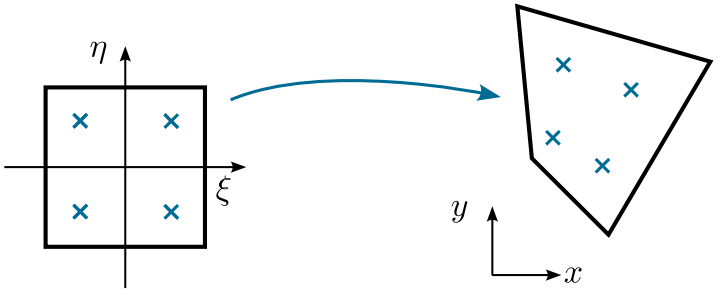
$$\nabla u = \sum_{i=1} \nabla N_i u_i, \quad \nabla v = \sum_{i=1} \nabla N_i v_i$$

$$\sum_{i=1}\sum_{j=1} \int_\Omega \nabla N_i u_i \cdot \nabla N_j v_j \, d\Omega = \sum_{i=1} \int_\Omega f\, N_i v_i \, d\Omega$$

$$K_{ij} = \int_\Omega \nabla N_i \cdot \nabla N_j \, d\Omega, \qquad f_i = \int_\Omega f\, N_i \, d\Omega$$

Element Loop Algorithm

Gauss Integration:



1D (per element):

$$K_{ab}^{(e)} \approx \sum_q w_q \left.\frac{dN_b}{dx}\right|_{\xi_q} \left.\frac{dN_a}{dx}\right|_{\xi_q} \left|J(\xi_q)\right|$$

$$f_a^{(e)} \approx \sum_q w_q f(x_q)\, N_a(\xi_q) \left|J(\xi_q)\right|$$

2D (per element):

$$K_{ab}^{(e)} \approx \sum_q w_q \left.\left(\nabla N_b \cdot \nabla N_a\right)\right|_{(\xi_q,\eta_q)} \left|\mathbf{J}(\xi_q,\eta_q)\right|$$

$$f_a^{(e)} \approx \sum_q w_q f(\mathbf{x}_q)\, N_a(\xi_q,\eta_q) \left|\mathbf{J}(\xi_q,\eta_q)\right|$$

Pseudocode: $\mathbf{K}^{(e)}$ and $\mathbf{f}^{(e)}$

Input: Element node coordinates $\mathbf{x}_a$,
Source term $f(x)$,
Gauss points and weights (ξ_q, η_q, w_q)
Output: Element stiffness matrix $\mathbf{K}^{(e)}$,
Element load vector $\mathbf{f}^{(e)}$
Initialize $\mathbf{K}^{(e)} \leftarrow \mathbf{0}$, $\mathbf{f}^{(e)} \leftarrow \mathbf{0}$
for each Gauss point q in the parent element **do**
  Evaluate
$N_a(\xi_q, \eta_q)$, $\nabla_\xi N_a(\xi_q, \eta_q)$
  Compute Jacobian:
$\mathbf{J} \leftarrow \sum_a \mathbf{x}_a \otimes \nabla_\xi N_a$
  Compute $|\mathbf{J}|$, and $\mathbf{J}^{-1}$
  Physical gradients:
$\nabla N_a \leftarrow \mathbf{J}^{-1} \nabla_\xi N_a$
  Physical point:
$\mathbf{x}_q \leftarrow \sum_a N_a(\xi_q, \eta_q)\, \mathbf{x}_a$
  for each local node a **do**
    for each local node b **do**
      $K_{ab}^{(e)} \leftarrow K_{ab}^{(e)} + w_q \left(\nabla N_b \cdot \nabla N_a\right) |\mathbf{J}|$
    end for
    $f_a^{(e)} \leftarrow f_a^{(e)} + w_q f(\mathbf{x}_q)\, N_a\, |\mathbf{J}|$
  end for
end for